

DAIRE CENTRAL SYSTEM

NMB Lender Subsystem

Demo Integration Guide for Lender Developers
Everything NMB needs to build, connect and pass the demo

Environment (demo)

Central System API: `http://172.17.16.76:8000`

Network: shared trusted LAN — **no API keys required**

Lender: NMB-001 · NMB Bank · COMMERCIAL_BANK

Currency: TZS · Reference borrower example: 1001

Version 1.0 · September 19, 2026 · Prepared by the DAIRE Central team
Contracts mirror LENDER_SUBSYSTEM_README.md in the DAIRE repository

Contents

1. Integration model — one trusted network, keyless JSON
2. What NMB must expose — GET /borrowers
3. Borrower response contract (exact JSON)
4. What NMB calls — push to Central
5. What NMB receives — credit result broadcast
6. Registration and end-to-end demo flow
7. Error behaviour, audit and security rules
8. Acceptance checklist
9. Borrower portal loan application — what the borrower fills in
10. Automatic bank data — what NMB supplies without borrower action
11. Merged borrower database — where every field lands in Central

1 · Integration model — one trusted network, keyless JSON

All DAIRE subsystems — the NMB lender subsystem, the DAIRE Central System, the AI engine and the blockchain bridge — run on the **same trusted network**. Inter-system calls are therefore plain HTTP + JSON **without API keys or bearer tokens**.

You do not need to implement any authentication for the demo. Send and accept plain JSON. Central is configured with `REQUIRE_API_KEYS=false`. If a subsystem is ever exposed beyond the LAN, Central switches that flag on and every call must then carry `X-API-Key` — build your receive endpoint so adding header checks later is a one-line change.

DIRECTION	CALL	AUTH FOR DEMO
Central → NMB	GET {nmb_base}/borrowers?borrower_reference=...	NONE
NMB → Central	POST http://172.17.16.76:8000/api/lender-data/receive/	NONE
Central → NMB	POST {nmb_base}/api/daire/central/receive/ (result broadcast)	NONE

2 · What NMB must expose — GET /borrowers

Central calls this endpoint to pull one borrower's data. Register your base URL (e.g. `http://172.17.16.70:9000`) with Central; Central appends the path itself.

```
GET /borrowers?borrower_reference=1001
Accept: application/json
```

Rules the endpoint must follow:

- Match **only** on the exact `borrower_reference` — return that customer's record, never another customer's.
- Return 200 with the JSON contract in section 3, or 404 with the body below when the reference is unknown to NMB. Never return an empty 200.
- Send `null` for unknown values — **never guess 0**. A guess overwrites a real value as "latest" during conflict resolution.
- Dates ISO YYYY-MM-DD, datetimes with Z, money as JSON numbers (max 4 decimals), currency TZS in `source_metadata.currency`.
- **All identifiers are plain numbers.** Customer ids are zero-padded digits — 001, 002, 003 (no prefixes like NMB-CUST-). Account, loan, repayment and transaction ids are digit-only too (3001, 4001, 5001, 6001). Send them as JSON numbers or digit-only strings — Central accepts both and normalizes.
- Support audit logging for every data pull (who asked, what was shared, when).

404 Not Found

```
{ "detail": "Borrower not found", "borrower_reference": "1001" }
```

3 · Borrower response contract (exact JSON)

HTTP 200 body — one object. This is the shape Central validates and merges; extra fields are kept in account metadata for audit but never scored.

```
{
  "borrower_reference": "1001",
  "customer_id": "001",
  "age": 29,
  "gender": "MALE",
  "employment_status": "EMPLOYED",
  "income": 1200000.0000,
  "business_information": {
    "business_name": "Amina Foods", "business_type": "RETAIL",
    "registration_number": "12345", "business_start_date": "2020-05-01"
  },
  "account_information": { "account_count": 2, "currency": "TZS", "customer_since": "2020-01-15" },
  "account_reference": "3001",
  "account_name": "Amina Juma",
  "transaction_frequency": 240,
  "income_frequency": 24,
  "savings": 850000.0000,
  "balance_stability": 0.82,
  "cash_flow_patterns": {
    "average_monthly_income": 1200000.0, "average_monthly_expenses": 700000.0,
    "income_months": 12, "active_months": 12, "trend_bps": 1250
  },
  "account_activity": {
    "first_transaction_date": "2024-01-03", "last_transaction_date": "2025-01-31",
    "active_months": 12, "transaction_count": 240
  },
  "transactions": [
    {
      "transaction_id": "6001",
      "account_reference": "3001",
      "transaction_date": "2025-01-05T10:30:00Z",
      "value_date": "2025-01-05",
      "type": "INCOME", "category": "SALARY",
      "description": "Monthly salary",
      "amount": 1200000.0, "currency": "TZS", "direction": "CREDIT",
      "balance_after": 1500000.0, "counterparty": "Employer Ltd", "status": "COMPLETED"
    }
  ],
  "balance_history": [
    { "date": "2025-01-31", "account_reference": "3001", "closing_balance": 850000.0, "currency": "TZS" }
  ],
  "loans": [
    {
      "loan_id": "4001",
      "loan_reference": "4001",
      "account_reference": "3001",
      "loan_amount": 500000.0,
      "loan_date": "2024-06-12",
      "loan_duration_months": 12,
      "interest_rate": 12.5,
      "outstanding_balance": 320000.0,
      "credit_limit": 500000.0,
      "status": "ACTIVE",
      "default_status": "CLEAR",
      "repayments": [
        {
          "repayment_id": "5001",
          "repayment_amount": 42000.0,
          "repayment_date": "2025-01-05",
          "due_date": "2025-01-05",
          "days_overdue": 0, "missed_payments": 0, "late_payments": 0,
          "default_status": "CLEAR", "status": "ON_TIME"
        }
      ]
    }
  ],
  "verification": {
    "identity_verified": true, "identity_provider": "NIDA", "identity_match_score": 1.0,
    "phone_verified": true, "account_owner_verified": true,
    "source_verified_at": "2025-02-01T08:00:00Z"
  }
}
```

```

},
"source_metadata": {
  "source_system": "NMB_CORE_BANKING", "source_version": "2025.1",
  "data_as_of": "2025-01-31T23:59:59Z", "currency": "TZS", "records_returned": 242
}
}

```

Field rules

FIELD	REQUIRED	MEANING
borrower_reference	YES	The central unique identifier requested by DAIRE — one digit off creates a second borrower and the lenders never combine.
customer_id	YES	NMB's internal customer identifier.
account_reference	Yes when available	Stable NMB account identifier — must not change between pushes.
transaction_frequency / income_frequency	YES	Counts in the selected reporting period.
loans	Yes when present	All current and historical loans; repayments nested under their loan.
transactions, balance_history	Recommended	Detailed rows for central off-chain processing.
verification	Recommended	Identity / phone / account-owner evidence.
source_metadata	YES	Provenance, version, data_as_of timestamp, currency.

Loan status ∈ ACTIVE, CURRENT, OPEN, COMPLETED, CLOSED, DEFAULTED, WRITTEN_OFF. Repayment status ∈ ON_TIME, LATE, MISSED, DEFAULTED. Never send PINs, passwords, card security codes or full tokens.

4 · What NMB calls — push to Central

NMB can push a borrower directly instead of waiting for a pull. Keyless — just JSON.

```
POST http://172.17.16.76:8000/api/lender-data/receive/
Content-Type: application/json
```

```
{
  "lender_id": "NMB-001",
  "borrower_reference": "1001",
  "account_reference": "3001",
  "payload": {
    "borrower_reference": "1001",
    "customer_id": "001",
    "transaction_count": 240,
    "transaction_total_amount": 18400000.0,
    "transaction_currency": "TZS",
    "income_count": 24,
    "income_total_amount": 14400000.0,
    "transaction_period_start": "2024-02-01",
    "transaction_period_end": "2025-01-31",
    "loans": []
  }
}
```

Send aggregates, not thousands of rows. `transaction_count` + `transaction_total_amount` (+ income counts/totals) are enough for scoring; Central derives the same aggregates automatically if a `transactions` array is sent instead. `payload.loans` must always be a list (`[]` when none) and the `borrower_reference` inside `payload` must equal the top-level one.

Success response (201)

```
{
  "status": "COMPLETED",
  "lender_id": "NMB-001",
  "borrower_reference": "1001",
  "summary": {
    "transaction_count": 240, "transaction_total_amount": 18400000.0,
    "transaction_currency": "TZS", "income_count": 24,
    "income_total_amount": 14400000.0,
    "total_loans_for_borrower": 1, "active_loans": 1
  },
  "borrower": { "...merged unified borrower record..." }
}
```

Conflicts are expected. When two lenders disagree on identity fields (age, income, ...), Central keeps both: latest push wins, the losing value is preserved in `data_conflicts`. Loans / repayments / transactions never conflict — they are keyed per lender + ID and added together. Push `null` when a value is unknown so a guess never overwrites a real value.

5 · What NMB receives — credit result broadcast

After AI and blockchain scoring finish, Central pushes the final credit result to every lender linked to the borrower — including NMB, at the fixed receive path below. Expose one POST handler there.

```
POST {nmb_base}/api/daire/central/receive/
Content-Type: application/json
```

```
{
  "result_type": "CREDIT_RESULT",
  "borrower_reference": "001",           // ← NMB's own customer id
  "central_borrower_reference": "1001", // central unified reference (audit only)
  "lender_customer_id": "001",
  "assessment_reference": "ASM-2026-0005",
  "result": {
    "kind": "BLOCKCHAIN_SCORE",
    "generated_at": "2026-09-18T10:35:00Z",
    "credit_score": 652,                  // public 350-800 scale
    "risk_band": "FAIR",
    "transaction_hash": "0xabc123...", "block_number": 152300
  },
  "results": {
    "ai": {
      "reputation": "EXCELLENT",
      "score": "0.9450",                 // decimal string — parse as decimal
      "risk_level": "LOW",
      "behavior_summary": "Borrower shows consistent on-time repayments...",
      "model_version": "credit-risk-joblib-v1"
    },
    "blockchain": {
      "credit_score": 652,
      "risk_band": "FAIR",
      "contract_address": "0xabc123...",
      "transaction_hash": "0xabc123...", "block_number": 152300
    }
  }
}
```

What your endpoint must do:

1. Look up your customer by `borrower_reference` — ignore results for references you do not hold (log the attempt, reply 404).
2. Store or display the result for that customer only. Central sends **derived scores only** — never raw transactions or balances.
3. Reply 200 with any JSON acknowledgement, e.g.:

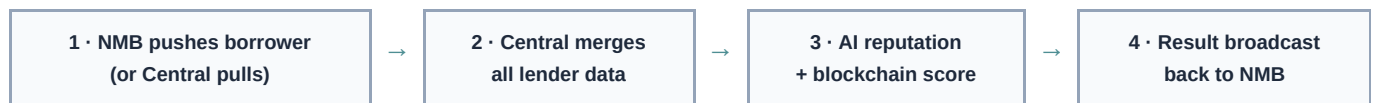
```
{ "received": true, "lender_system": "NMB", "result_type": "CREDIT_RESULT", "score": 652 }
```

6 · Registration and end-to-end demo flow

Before the demo, the DAIRE team registers NMB once (keyless — no key exchange needed):

```
POST http://172.17.16.76:8000/api/lenders/
{
  "lender_id": "NMB-001",
  "institution_name": "NMB Bank",
  "institution_type": "COMMERCIAL_BANK",
  "api_base_url": "http://172.17.16.70:9000",    // NMB demo host
  "api_status": "CONNECTED"
}
```

Demo flow



1. **Data in.** NMB pushes POST `/api/lender-data/receive/` — or Central pulls POST `/api/borrowers/pull-from-all-lenders/` `{"borrower_reference": "1001"}`, which calls GET `{nmb_base}/borrowers?borrower_reference=...`
2. **Merge.** Central normalizes the payload into the unified borrower (accounts, loans, repayments, financial profile) and records a DataExchange audit entry.
3. **Score.** An assessment opens on the merged borrower; the AI engine returns a reputation (0–1) and the smart contract records the credit score (350–800) on-chain.
4. **Result out.** Central broadcasts the combined result to NMB's `/api/daire/central/receive/` endpoint.

Test utilities on Central (use before wiring the real NMB host)

URL ON 172.17.16.76:8000	PURPOSE
GET <code>/api/mock-lender/NMB/borrowers?borrower_reference=1001</code>	Mock lender pull target with deterministic demo data — lets you rehearse "pull from all lenders" before NMB's endpoint is live.
POST <code>/api/mock-lender/NMB/api/daire/central/receive/</code>	Mock broadcast receiver — rehearse the result broadcast before NMB implements theirs.
GET <code>/api/data-exchanges/</code>	Every push/pull/broadcast is logged here (payload + status) for reconciliation.

7 · Error behaviour, audit and security rules

SITUATION	NMB RETURNS
Borrower unknown at NMB	404 + { "detail": "Borrower not found", "borrower_reference": "..." }
Malformed request (missing reference)	400
Temporary outage / dependency down	503
Later, if keys are switched on: missing/invalid credential	401

- **Never** return a successful response containing a different customer's records.
- Verify consent / legal basis before releasing data; log an audit event for every pull (borrower reference, requester, purpose, fields shared, timestamps, correlation id).

- Duplicate pushes are safe — Central keys records by `lender_id` + stable IDs, so re-sending the same borrower does not create duplicates (as long as `loan_id` / `account_reference` are stable).
- Use TLS when traffic leaves the trusted LAN, keep service accounts least-privileged, and never commit credentials or customer data to source control.

8 • Acceptance checklist

✓	REQUIREMENT
☐	GET <code>/borrowers?borrower_reference=...</code> returns the exact JSON contract of section 3 (or the aggregate push of section 4 is implemented instead).
☐	Unknown borrower → 404; never another customer's data.
☐	All dates ISO; all money numeric with TZS in <code>source_metadata.currency</code> .
☐	Stable <code>borrower_reference</code> , <code>account_reference</code> , <code>loan_id</code> across pushes; repayments nested under their loan.
☐	Loans always a list; <code>payload.borrower_reference</code> equals the top-level reference; null for unknowns, never 0.
☐	POST <code>/api/daire/central/receive/</code> exposed, matches on own <code>borrower_reference</code> , stores/validates the result, replies 200.
☐	Keyless calls work end-to-end against <code>http://172.17.16.76:8000</code> on the demo LAN.
☐	Audit event written for every data pull and every received broadcast.
☐	Duplicate requests safe; no credentials or secret customer security data in any response.

Before the demo: run the contract test — pull one borrower through `/api/borrowers/pull-from-all-lenders/` with NMB registered, confirm the merged record on the Data Exchange page, and confirm the broadcast acknowledgement appears in NMB's logs. One rehearsal avoids every integration surprise on demo day.

9 · Borrower portal loan application — what the borrower fills in

When a borrower applies for a loan in the borrower portal, they complete only **identity, declaration, loan request and consent**. They never re-type their financial history — that arrives automatically from the bank (section 10).

GROUP	FIELDS THE BORROWER PROVIDES
Identity	NIDA number, full name, date of birth (→ age), gender, phone (OTP-verified), email.
Employment	Employment status (EMPLOYED / SELF_EMPLOYED / BUSINESS_OWNER / UNEMPLOYED), employer name, declared monthly income.
Business (if any)	Business name, type (RETAIL / AGRICULTURE / SERVICES ...), BRELA registration number, business start date.
Loan request	Requested amount, purpose (BUSINESS / EDUCATION / AGRICULTURE / HOME ...), duration in months, preferred repayment date.
Consent	Explicit consent for DAIRE to pull data from lenders — purpose + expiry (stored in the existing Consent model).
Documents (optional in demo)	ID scan, payslip, bank statement.

10 · Automatic bank data — what NMB supplies without borrower action

The moment the borrower applies, their real financial behaviour flows in from NMB automatically — exactly the data contract of sections 2–4. The borrower does nothing for this part.

DATA	WHERE IT COMES FROM AT THE BANK
Verified identity (NIDA match, customer_id)	Bank KYC records.
account_reference, account name, customer-since	Core banking account.
Transactions (frequency, income frequency, amounts, counterparties)	Account statement feed.
Balance history + balance_stability	Daily/monthly closing balances.
Cash-flow patterns (average income/expenses, trend)	Computed by the bank from statements.
Savings	Deposit products.
Loans + repayments (amounts, dates, status, defaults, overdue days)	Loan management system.
verification + source_metadata (as-of timestamp, version)	Bank compliance layer.

11 · Merged borrower database — where every field lands in Central

Central merges both sources into one unified borrower record. Matching is only on exact borrower_reference; financial records are additive (keyed per lender + stable IDs); identity disagreements are logged in data_conflicts (latest push wins) and never silently lost.

CENTRAL MODEL / FIELD	FROM BORROWER PORTAL	FROM BANK(S)
Borrower — reference, name, age, gender	✓ reference + name	✓ verified identity, age, gender
Borrower.income	declared (fallback)	✓ actual — overrides declaration
Borrower.employment_status	self-declared	✓ verified
Borrower.business_information	✓ self-declared	✓ BRELA-verified when available
BorrowerAccount (per lender)	—	✓
BorrowerLoan + RepaymentRecord	— (the portal's loan <i>request</i> is separate)	✓ history only
BorrowerFinancialProfile (frequency, stability, savings)	—	✓ derived by Central
Consent	✓	—
data_conflicts	—	auto — bank wins on financials; disagreement is logged

Demo note: Central has no LoanApplication model — the loan *request* lives in the portal/bank. A portal app uses the existing endpoints (register borrower → receive/pull → assessment → broadcast) and keeps the application form on its side. The final result reaches the borrower via the broadcast in section 5, and the borrower sees the reputation + 350–800 score in the portal.

End-to-end with the portal



Questions → DAIRE Central team · Central API on the demo LAN: <http://172.17.16.76:8000> · Contracts: LENDER_SUBSYSTEM_README.md, AI_DATA_CONTRACT.md